Generative AI for Synthetic Financial Data

Instructor: Chao ZHANG

March 25, 2025

Reinforcement Learning Recap

Introduction to Generative AI
- Autoencoders
- Generative Adversarial Networks (GAN)

Applications in Finance

Challenges of Generative AI in Finance

Reinforcement Learning Algos

- Value-based methods
  - focus on estimating *a value function (usually the Q-function)* and then deriving a policy from it (typically by choosing the action with the highest estimated value).
  - Q-Learning (Off-policy)
  - SARSA (On-policy)
- Policy-based methods
  - focus directly on *learning the policy (i.e., the mapping from states to actions)* without explicitly maintaining a value function.
  - REINFORCE
- Hybrid Approaches: Actor-Critic
- Examples: Super Mario Bros, Mario Cart

RL in Finance

Why RL has achieved remarkable success in games, but not finance?

What differences between games and finance?

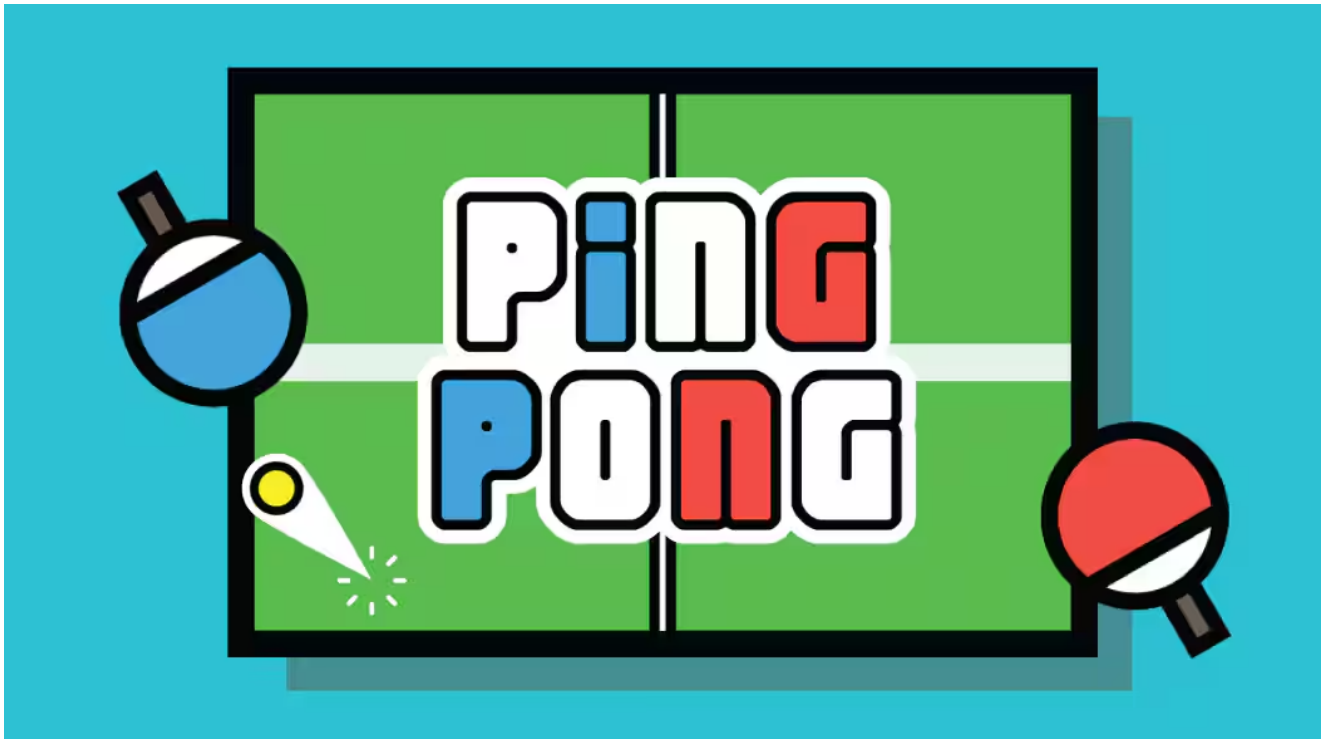
RL in Finance

Why RL has achieved remarkable success in games, but not finance?
What differences between games and finance?

- finance: noisy and non-stationary, multi-agent
- games: well-defined, clear metrics
- In finance, performance must account for transaction costs, market impact, risk, and other confounding factors, complicating both the training and evaluation of RL algorithms.

1. Environment Complexity and Stationarity

- Games:
  - Well-defined, consistent rules.
  - Stationary dynamics enable stable learning.
- Finance:
  - Highly non-stationary; market conditions change unpredictably.
  - External factors (news, economic events) add complexity.



2. Simulation and Training Data

- Games:
  - High-fidelity simulators allow abundant, controlled training.
  - Agents can play millions of games rapidly.
- Finance:
  - Lack of perfect simulators; historical data may not capture agent interactions.
  - Live trading data is limited and costly; risk of real-world losses.

3. Reward Structure and Objectives

- Games:
  - Clear, immediate, and well-defined rewards (e.g., points, wins).
- Finance:
  - Noisy, delayed, and multi-objective rewards (profit vs. risk vs. market impact).
  - Difficult to design reward signals that capture true trading performance.

4. Adversarial Dynamics and Risk Management

- Games:
  - Opponents' strategies are either fixed or modeled within the game.
  - Risk is limited to in-game performance without external consequences.
- Finance:
  - Markets involve many adaptive agents with hidden strategies.
  - High financial risk and regulatory constraints demand robust risk management.

5. Real-Time Constraints and Evaluation Complexity

- Real-Time Decision Constraints:
  - Financial markets demand ultra-low latency decisions.
  - RL algorithms that work well in slower game environments may struggle with these speed requirements.
- Evaluation Complexity:
  - Performance metrics in finance must account for transaction costs, risk, and market impact.
  - Unlike game scores, financial rewards are noisy and multi-dimensional.

Reinforcement Learning Recap

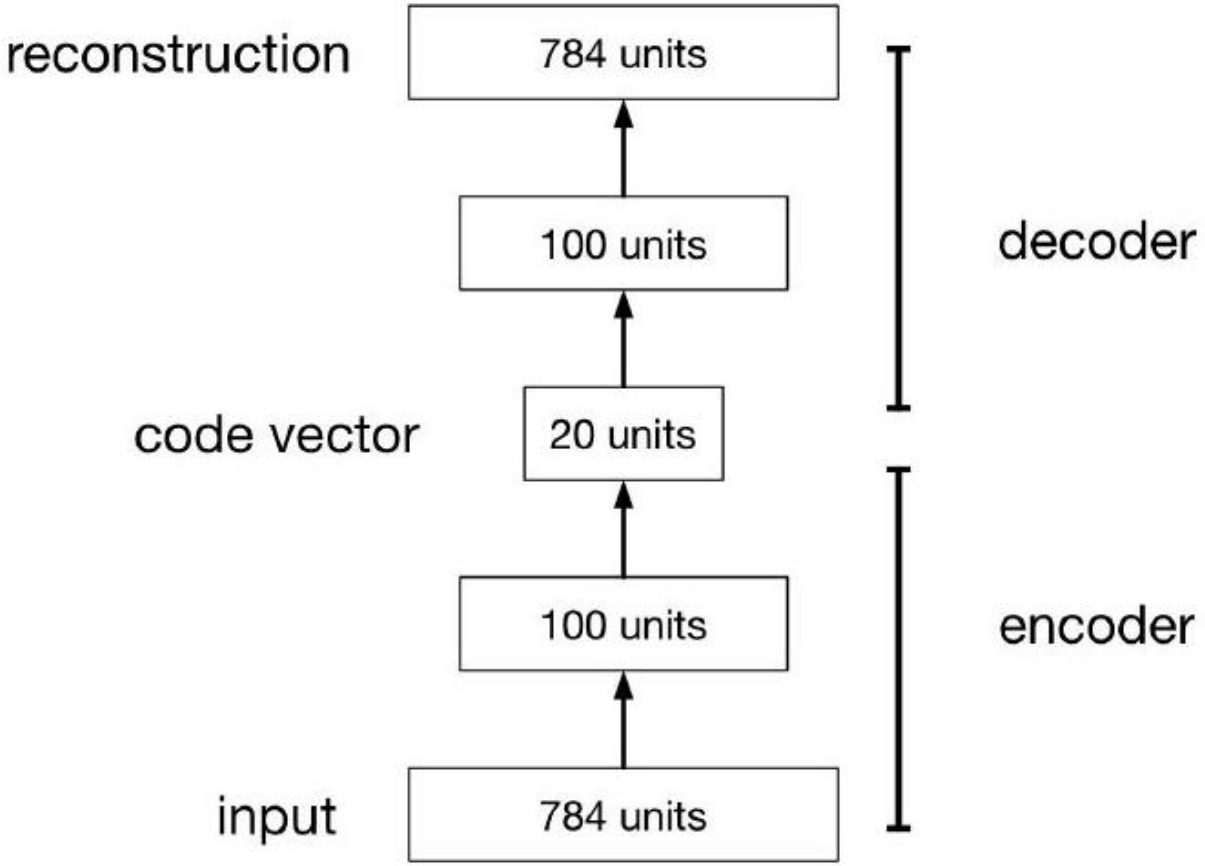
Introduction to Generative AI
- Autoencoders
- Generative Adversarial Networks (GAN)

Applications in Finance

Challenges of Generative AI in Finance

Autoencoders

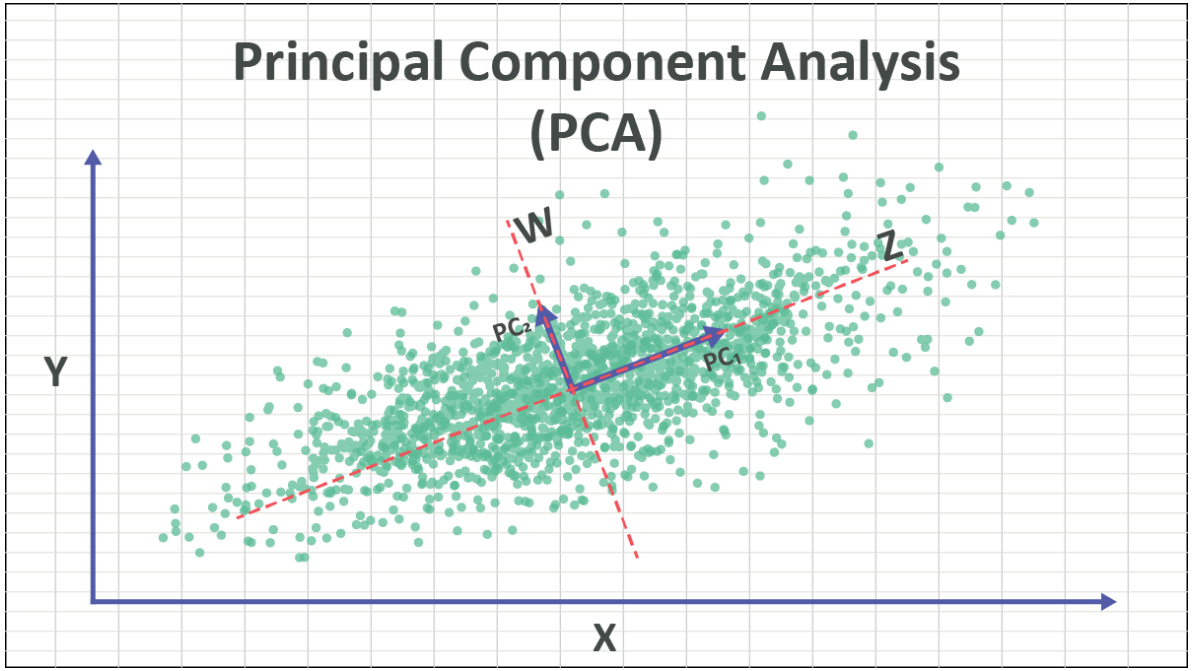
- An autoencoder is a feed-forward neural net whose job it is to take an input **x** and predict **x**.
- To make this non-trivial, we need to add a bottleneck layer whose dimension is much smaller than the input.



Why autoencoders?

- Map high-dimensional data to two dimensions for visualization
- Compression (i.e. reducing the file size)
- Learn abstract features in an unsupervised way so you can apply them to a supervised task
- Unlabled data can be much more plentiful than labeled data
- Learn a semantically meaningful representation where you can, e.g., interpolate between different images.

Principal Component Analysis



PCA seeks to find the principal components (PCs) that capture the maximum variance in the data. Mathematically:

$$\min_{W} \sum_{i} \left\| x_i - WW^T x_i \right\|^2$$

subject to $W^T W = I$, ensuring that the PCs are orthonormal.

Orthogonal Decomposition via Pythagorean Theorem

For each x_i, we can decompose:

$$x_i = WW^T x_i + \left(x_i - WW^T x_i\right)$$

Since $WW^T x_i$ is the projection and the residual is orthogonal:

$$\|x_i\|^2 = \|WW^T x_i\|^2 + \|x_i - WW^T x_i\|^2.$$

Noting that $\|WW^T x_i\|^2 = \|W^T x_i\|^2$ because W has orthonormal columns.

Summing Over Data Points

Summing the above identity over all data points:

$$\sum_{i=1}^{n} \|x_i\|^2 = \sum_{i=1}^{n} \|W^T x_i\|^2 + \sum_{i=1}^{n} \|x_i - WW^T x_i\|^2.$$

Since $\sum_{i=1}^{n} \|x_i\|^2$ is a constant (total energy of the data), we have:

$$\sum_{i=1}^{n} \|x_i - WW^T x_i\|^2 = \text{constant} - \sum_{i=1}^{n} \|W^T x_i\|^2.$$

Equivalence of Objectives

Reconstruction Error Minimization

$$\min_{W} \sum_{i=1}^{n} \|x_i - WW^T x_i\|^2$$

is equivalent to

Variance Maximization

$$\max_{W} \sum_{i=1}^{n} \|W^T x_i\|^2,$$

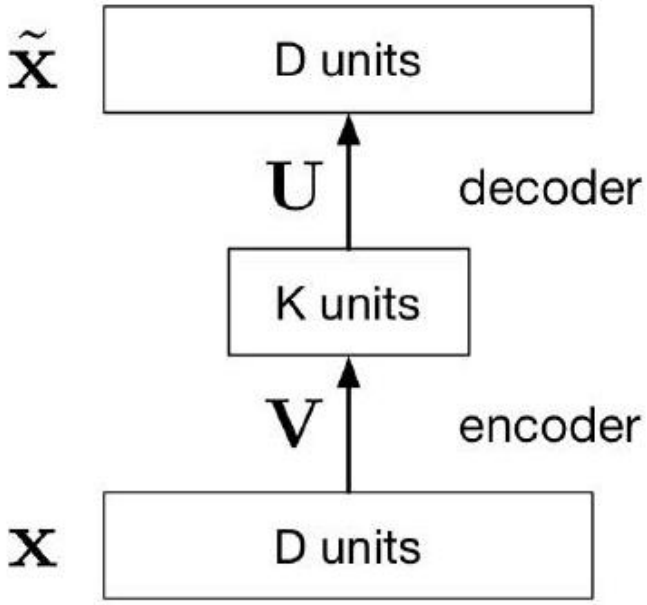
because the total energy $\sum_i \|x_i\|^2$ is fixed.

Principal Component Analysis

- The simplest kind of autoencoder has one hidden layer, linear activations, and squared error loss.

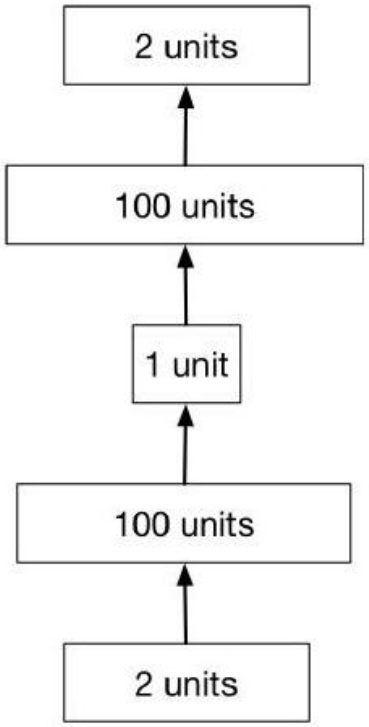
$$\mathcal{L}(\mathbf{x}, \tilde{\mathbf{x}}) = \|\mathbf{x} - \tilde{\mathbf{x}}\|^2$$

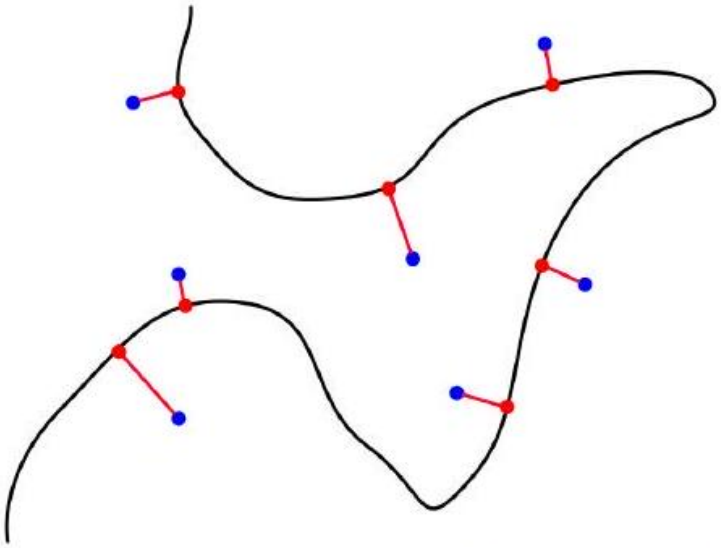
- This network computes $\tilde{\mathbf{x}} = \mathbf{UVx}$, which is a linear function.
- Suppose $K < D$: V maps $\mathbf{x}$ to a K-dimensional space, so it's doing dimensionality reduction.
- The output must lie in a K-dimensional subspace, namely the column space of $\mathbf{U}$.



Deep Autoencoders

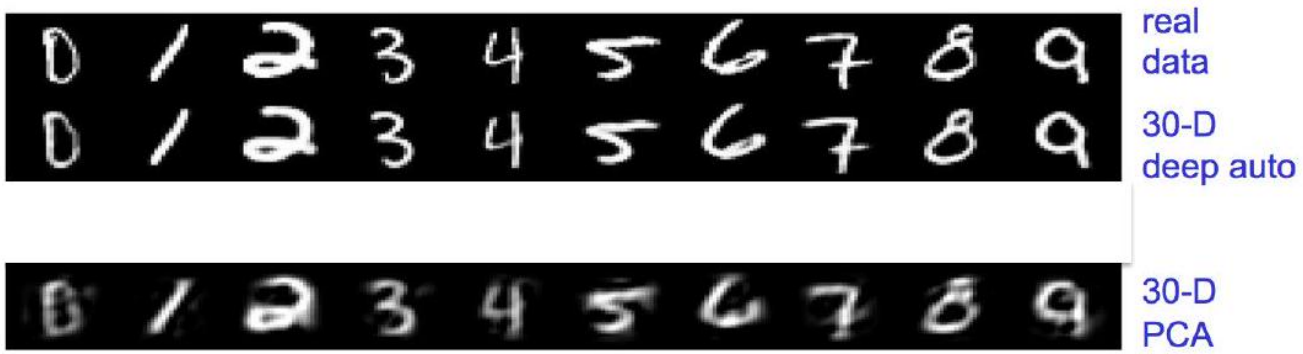
- Deep nonlinear autoencoders learn to project the data, not onto a subspace, but onto a nonlinear manifold
- This manifold is the image of the decoder.
- This is a kind of nonlinear dimensionality reduction.





Deep Autoencoders

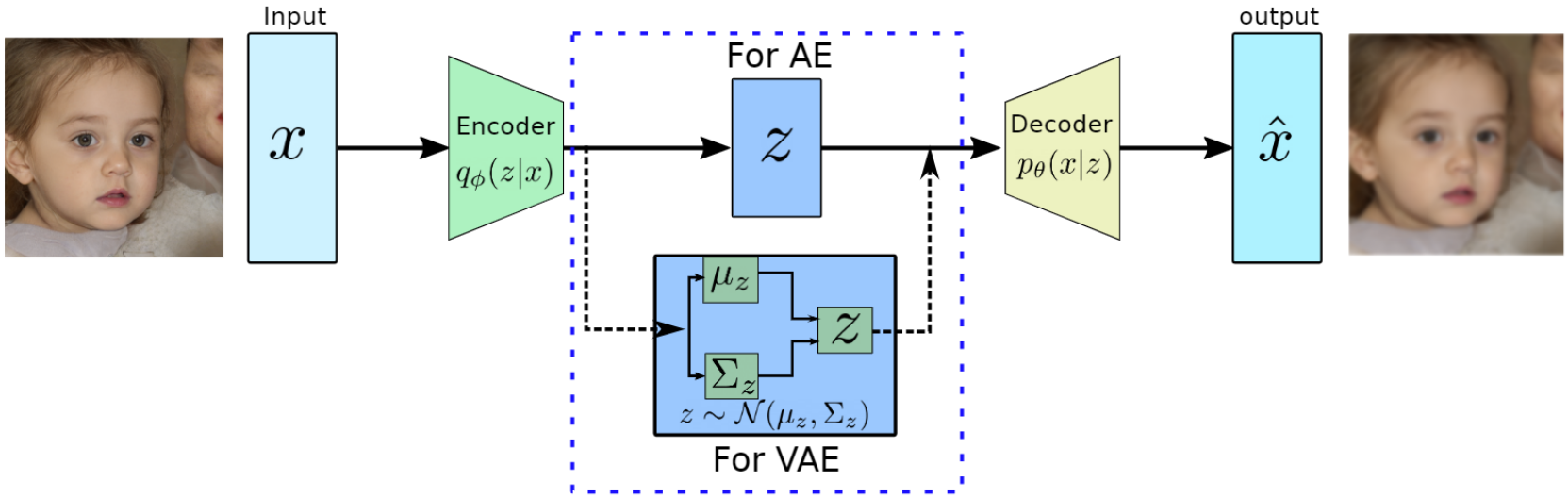
- Nonlinear autoencoders can learn more powerful codes for a given dimensionality, compared with linear autoencoders (PCA)



Limitations of Deep Autoencoders

- Focus on minimizing reconstruction loss, but the latent space may not be meaningful or interpretable
  - For example, it may simply copy pixel-level information without understanding object boundaries or textures.
- They're not generative models, so they don't define a distribution
- Memorization Risk: Autoencoders tend to memorize training data rather than learning generalizable features.
- The choice of architecture significantly impacts performance.
  - How to choose the latent dimension?

VAE


Input
x
Encoder
$q_\phi(z|x)$
For AE
z
μ_z
Σ_z
z
$z \sim \mathcal{N}(\mu_z, \Sigma_z)$
For VAE
Decoder
$p_\theta(x|z)$
output
$\hat{x}$

Observation Model

A probabilistic model describes how the observed data is generated from latent variables.

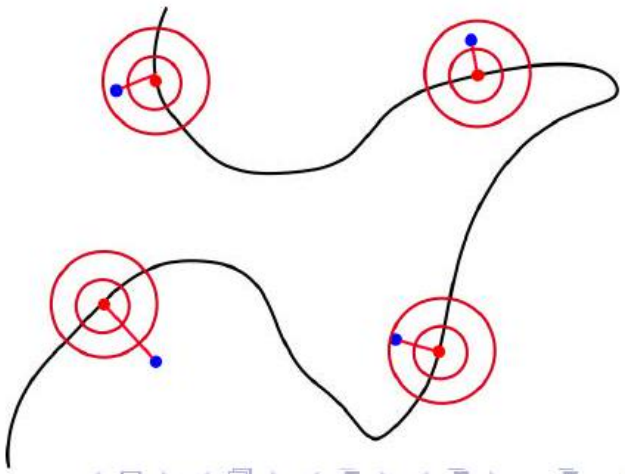
Consider training a generator with maximum likelihood.

$$\underbrace{p(\mathbf{x})}_{\text{actual data space}} = \int \underbrace{p(\mathbf{z})}_{\text{latent space}} \underbrace{p(\mathbf{x} \mid \mathbf{z})}_{\text{conditional prob}} \, \mathrm{d}\mathbf{z}$$

Generator modeled as a neural network

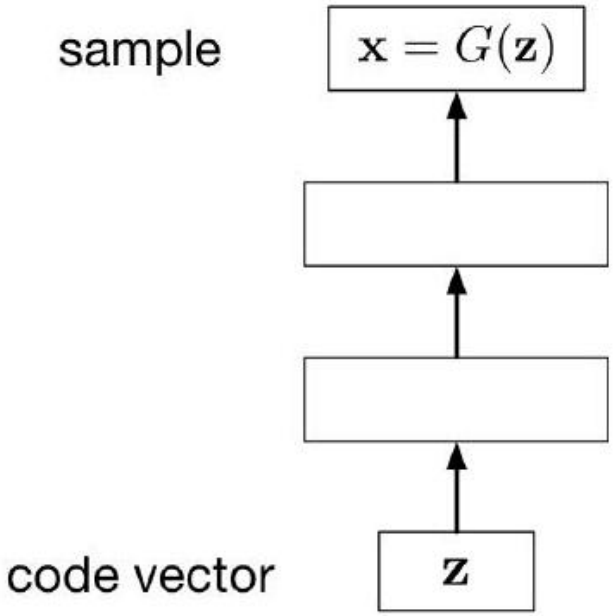
$$p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}\left(\mathbf{x}; G_{\boldsymbol{\theta}}(\mathbf{z}), \eta \mathbf{I}\right)$$

where $G_{\boldsymbol{\theta}}$ is the function computed by the decoder with parameters $\boldsymbol{\theta}$.



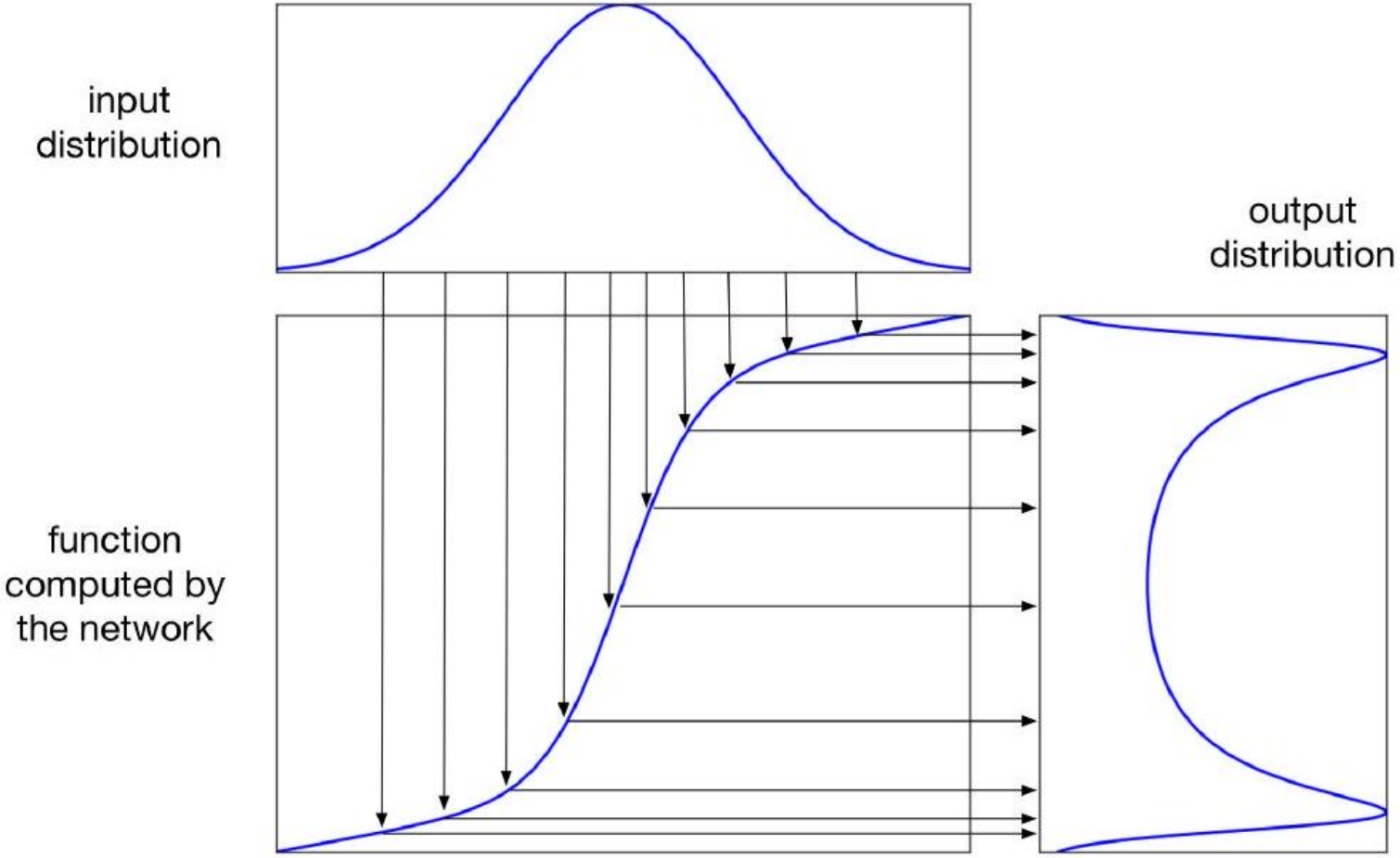
Implicit Generative Models

- Implicit generative models implicitly define a probability distribution
- Start by sampling the code vector z from a fixed, simple distribution (e.g. spherical Gaussian)
- The generator network computes a differentiable function G mapping $\mathbf{z}$ to an $\mathbf{x}$ in data space

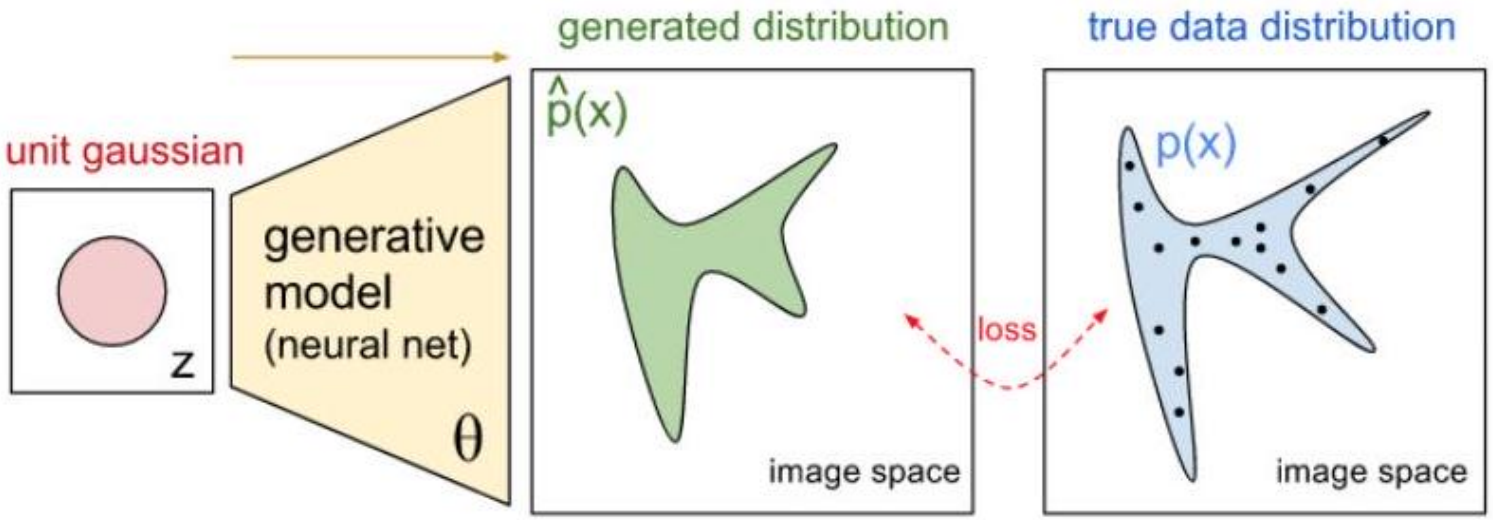


Implicit Generative Models

A 1-dimensional example:



Implicit Generative Models



VAE target

- VAEs aim to maximize the likelihood of the data $p(x)$.

$$p(x) = \int p(x|z)p(z)\,dz$$

- Direct maximization of $p(x)$ is intractable.
- This intractability arises from integrating over the latent space z.

Challenges:

- High Dimensionality
  - Evaluating the integral over a high-dimensional space z is computationally expensive.
- Non-Analytical Forms
  - integral does not have a closed-form solution.
- Computational Cost of numerical integration.

Variational Inference

- We obtain the lower bound using Jensen's Inequality: for a convex function h of a random variable X,

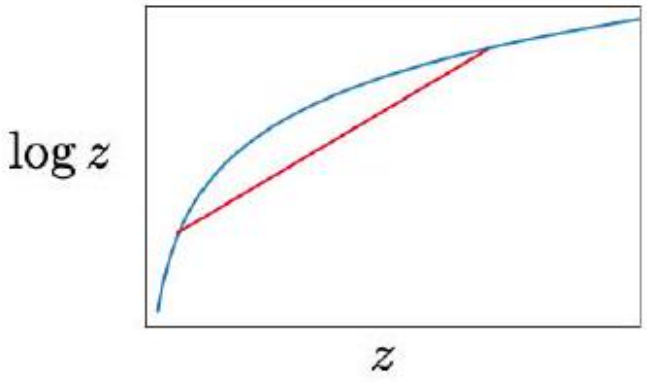
$$\mathbb{E}[h(X)] \geq h(\mathbb{E}[X])$$

- if h is concave (i.e. $-h$ is convex),

$$\mathbb{E}[h(X)] \leq h(\mathbb{E}[X])$$

Example: The function $\log z$ is concave/

$$\mathbb{E}[\log X] \leq \log \mathbb{E}[X]$$



Variational Inference

- Suppose we have some distribution $q(z)$. (We'll see later where this comes from.)
- We use Jensen's Inequality to obtain the lower bound.

$$
\begin{aligned}
\log p(\mathbf{x}) &= \log \int p(\mathbf{z}) p(\mathbf{x} \mid \mathbf{z}) \mathrm{d}\mathbf{z} \\
&= \log \int q(\mathbf{z} \mid \mathbf{x}) \frac{p(\mathbf{z})}{q(\mathbf{z} \mid \mathbf{x})} p(\mathbf{x} \mid \mathbf{z}) \mathrm{d}\mathbf{z} \\
&\geq \int q(\mathbf{z} \mid \mathbf{x}) \log \left[\frac{p(\mathbf{z})}{q(\mathbf{z} \mid \mathbf{x})} p(\mathbf{x} \mid \mathbf{z}) \right] \mathrm{d}\mathbf{z} \qquad \text{(Jensen's Inequality)} \\
&= \mathbb{E}_q \left[\log \frac{p(\mathbf{z})}{q(\mathbf{z} \mid \mathbf{x})} \right] + \mathbb{E}_q [\log p(\mathbf{x} \mid \mathbf{z})]
\end{aligned}
$$

$\Rightarrow$ Evidence Lower BOund (ELBO)

Variational Inference

- The ELBO provides a tractable lower bound on $\log p(x)$.
- The first term we'll look at is $\mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})]$
- Since we assumed a Gaussian observation model,

$$
\begin{aligned}
\log p(\mathbf{x} \mid \mathbf{z}) &= \log \mathcal{N}\left(\mathbf{x}; G_\theta(\mathbf{z}), \eta \mathbf{I}\right) \\
&= \log \left[\frac{1}{(2\pi\eta)^{D/2}} \exp\left(-\frac{1}{2\eta}\left\|\mathbf{x} - G_\theta(\mathbf{z})\right\|^2\right)\right] \\
&= -\frac{1}{2\eta}\left\|\mathbf{x} - G_\theta(\mathbf{z})\right\|^2 + \text{ const}
\end{aligned}
$$

- So this term is the expected squared error in reconstructing $\mathbf{x}$ from $\mathbf{z}$. We call it the reconstruction term.
- Through the reconstruction term, the model learns to generate data that closely resembles the original input.

Variational Inference

- The second term is $\mathbb{E}_q\left[\log \frac{p(z)}{q(\mathbf{z}|\mathbf{x})}\right]$.
- This is just $-\mathrm{D}_{\mathrm{KL}}(q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z}))$, where D_{KL} is the Kullback-Leibler (KL) divergence

$$\mathrm{D}_{\mathrm{KL}}(q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z})) \triangleq \mathbb{E}_q\left[\log \frac{q(\mathbf{z} \mid \mathbf{x})}{p(\mathbf{z})}\right]$$

- KL divergence is a widely used measure of distance between probability distributions, though it doesn't satisfy the axioms to be a distance metric.
- Typically, $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$. Hence, the KL term encourages q to be close to $\mathcal{N}(\mathbf{0}, \mathbf{I})$.
- regularizing the latent space and encouraging it to capture meaningful, generalizable features.

Variational Inference

- Hence, we're trying to maximize the variational lower bound, or variational free energy:

$$\log p(\mathbf{x}) \geq \mathcal{F}(\boldsymbol{\theta}, q) = \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})] - \mathrm{D}_{\mathrm{KL}}(q \| p)$$

- The term "variational" is a historical accident: "variational inference" used to be done using variational calculus, but this isn't how we train VAEs.
- **We'd like to choose q to make the bound as tight as possible.**

Tractable Family of Distributions

Common Choice: Gaussian Distribution

- Assume $q(z|x) = \mathcal{N}\big(z; \mu(x), \mathrm{diag}(\sigma^2(x))\big)$.
- Provides a closed-form KL divergence with the prior.
- Benefits:
  - Easy sampling.
  - Tractable computations.

Expressiveness:

- A simple Gaussian might not capture all complexities of $p(z|x)$.
- Alternatives include mixtures of Gaussians or using normalizing flows.

Reparameterization Trick

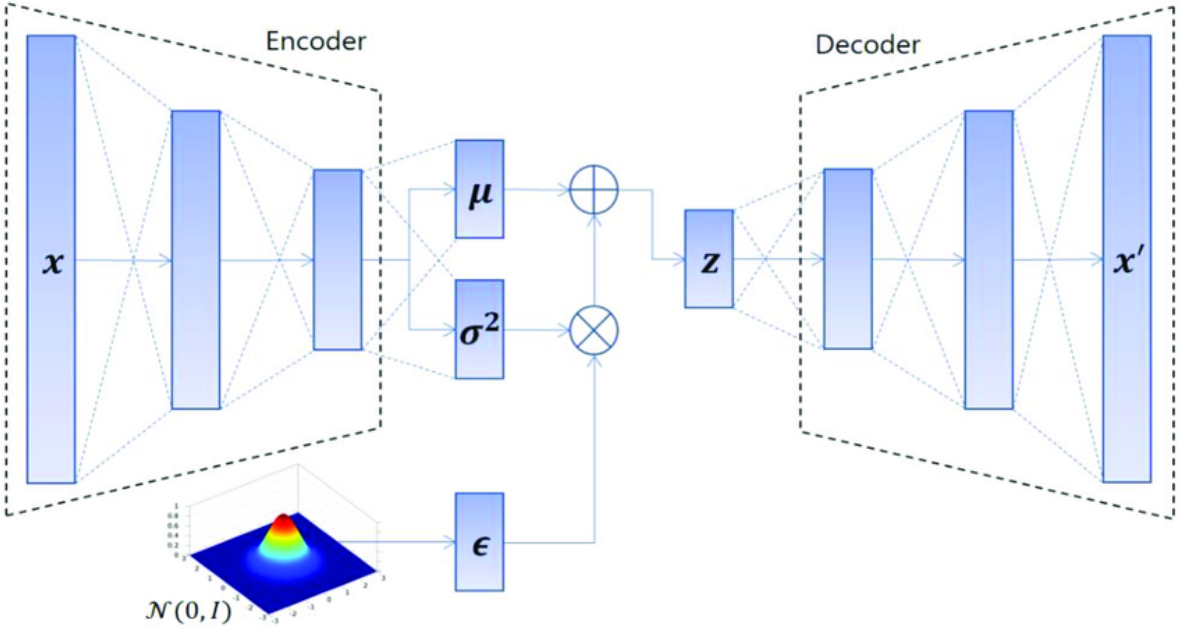
- VAE approximates $p(z \mid x)$ using a variational distribution $q(z \mid x)$, parameterized by a neural network (encoder).
- Directly sampling from $q(z|x)$ is not differentiable.
- Use the reparameterization trick to make it differentiable:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

- Enables backpropagation and efficient learning.

VAE Structure

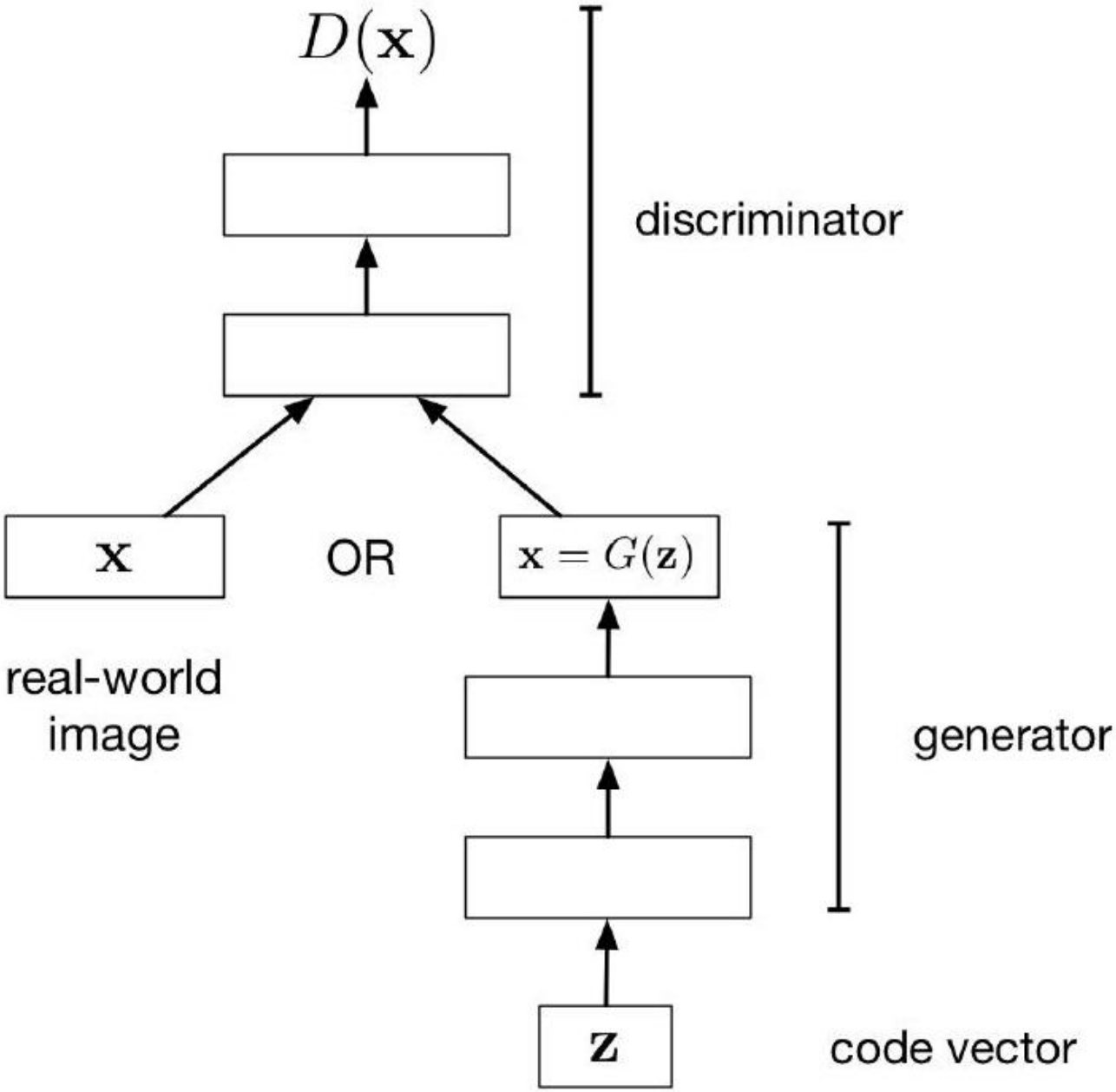
- **Encoder:** Maps input data x to a latent distribution $q(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mu, \sigma^2)$.
- **Latent Space:** Regularized to follow a standard normal distribution.
- **Decoder:** Generates data from sampled latent variables z according to $p(\mathbf{x} \mid \mathbf{z})$.



Generative Adversarial Networks

- The advantage of implicit generative models: if you have some criterion for evaluating the quality of samples, then you can compute its gradient with respect to the network parameters, and update the network's parameters to make the sample a little better
- The idea behind Generative Adversarial Networks (GANs): train two different networks
- The generator network tries to produce realistic-looking samples
- The discriminator network tries to figure out whether an image came from the training set or the generator network
- The generator network tries to fool the discriminator network

GAN


$D(\mathbf{x})$
discriminator
$\mathbf{x}$
OR
$\mathbf{x} = G(\mathbf{z})$
real-world image
generator
$\mathbf{z}$
code vector

Generative Adversarial Networks

- Let D denote the discriminator's predicted probability of being data
- Discriminator's cost function: cross-entropy loss for task of classifying real vs. fake images

$$\mathcal{J}_D = \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}[-\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[-\log(1 - D(G(\mathbf{z})))]$$

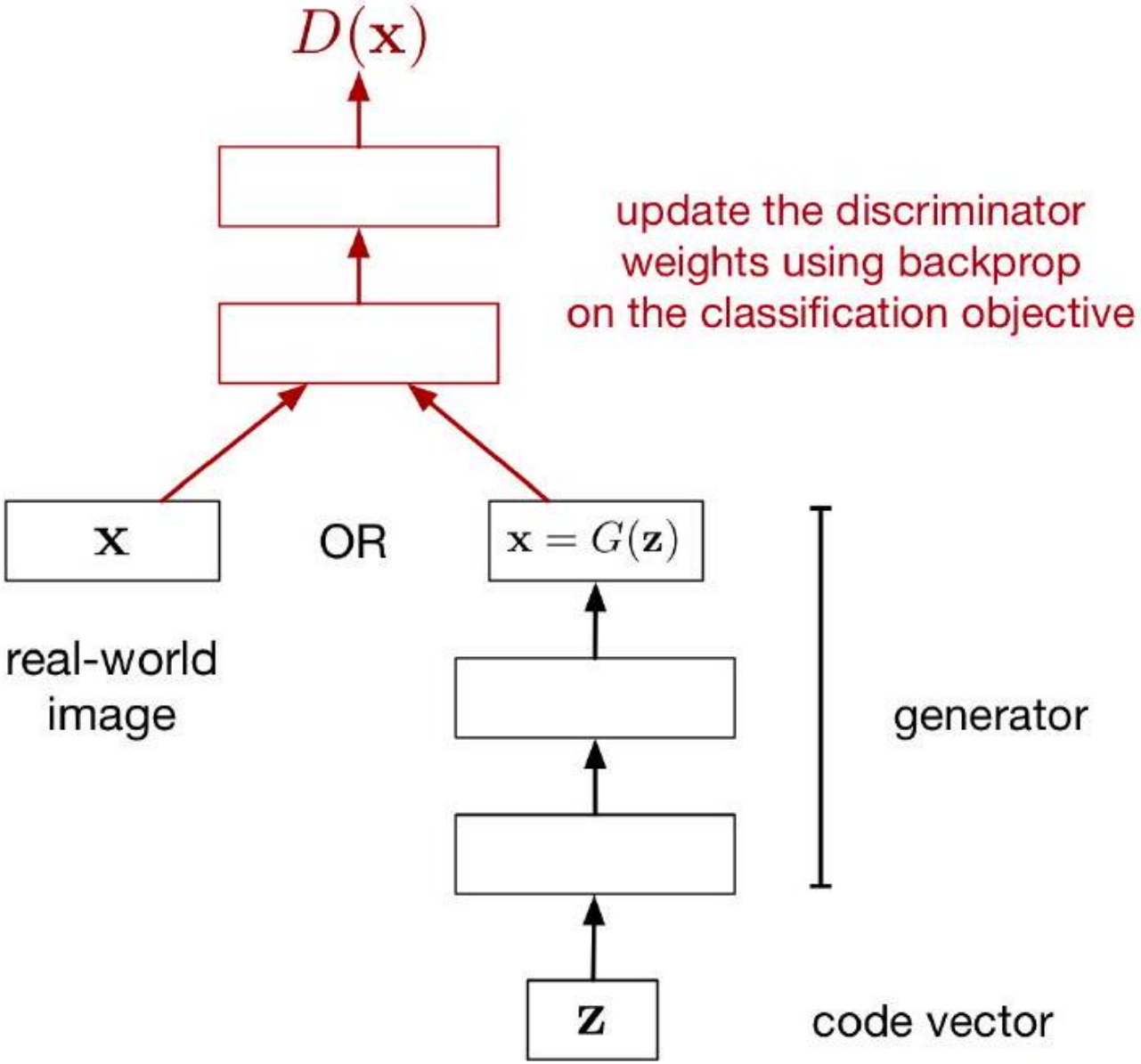
- One possible cost function for the generator: the opposite of the discriminator's

$$\begin{aligned}\mathcal{J}_G &= -\mathcal{J}_D \\ &= \text{ const } + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]\end{aligned}$$

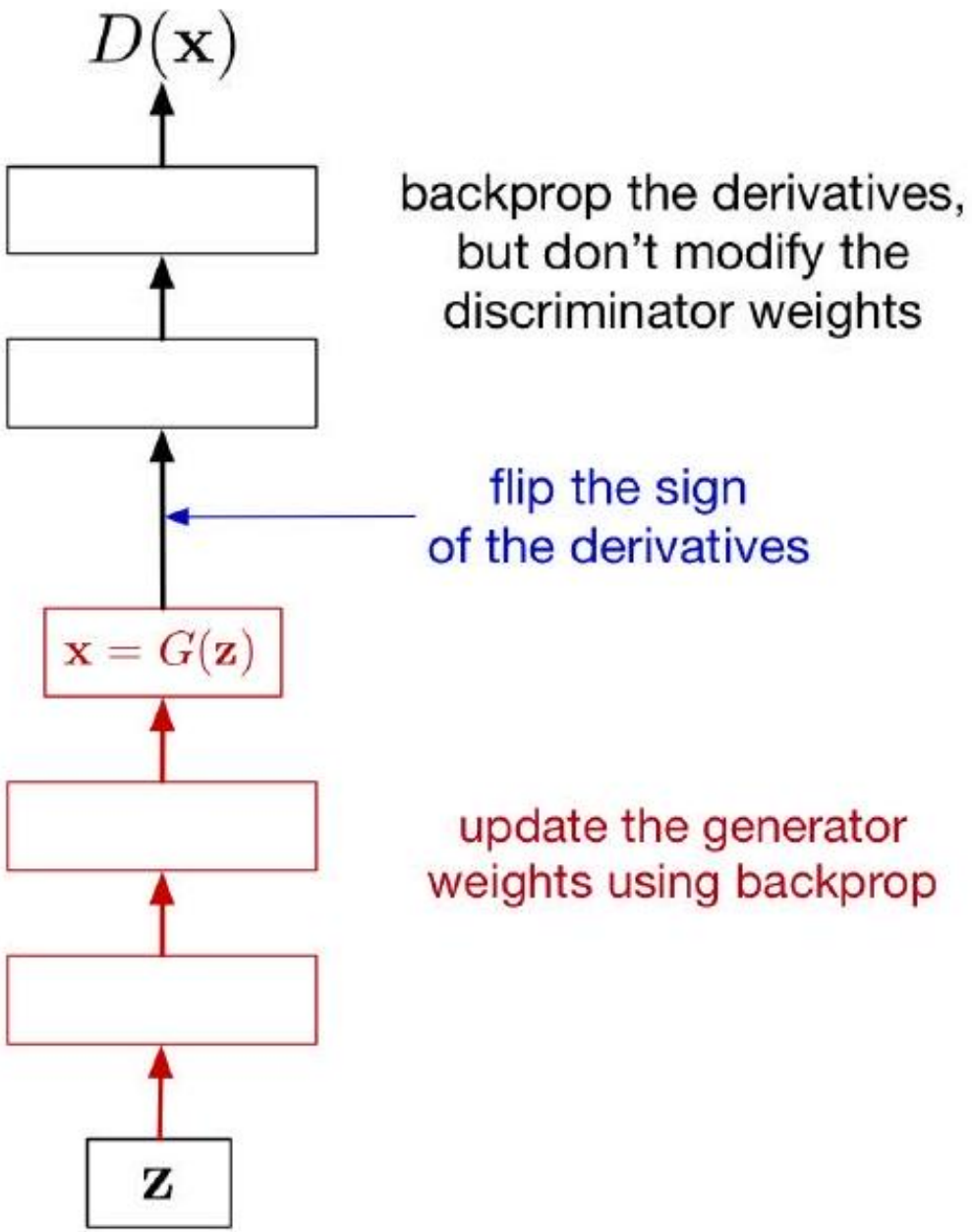
- This is called the minimax formulation, since the generator and discriminator are playing a zero-sum game against each other: $\max_G \min_D \mathcal{J}_D$

Generative Adversarial Networks

Updating the discriminator:



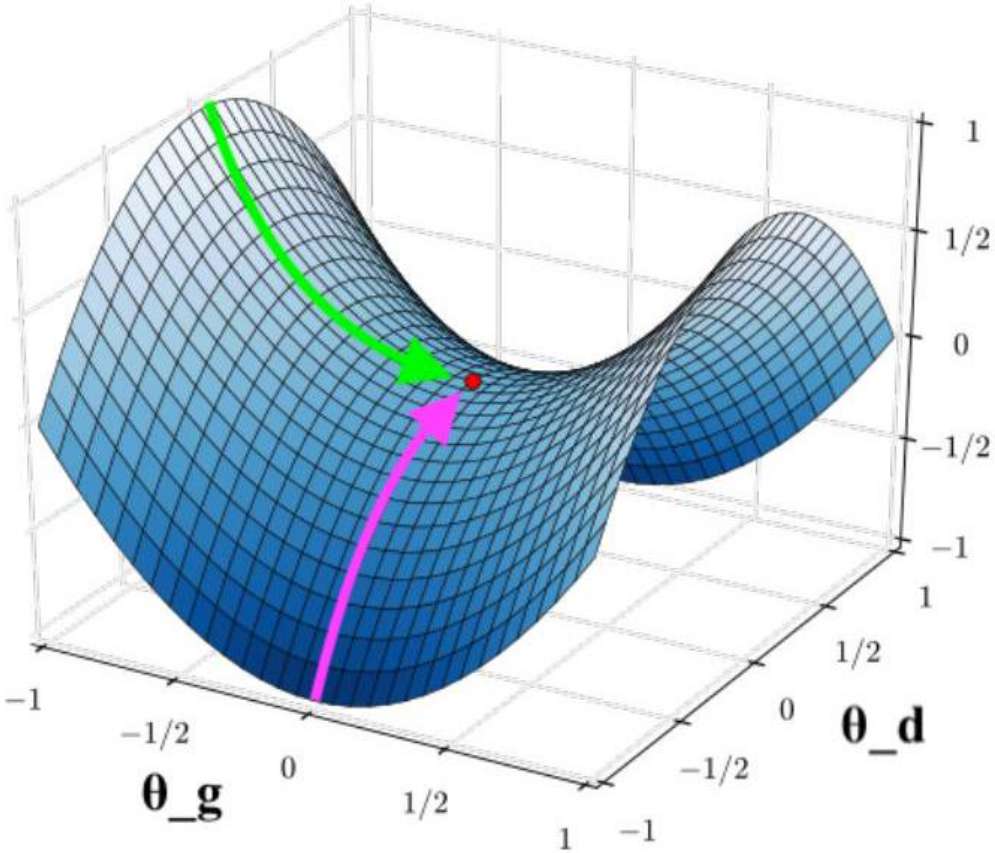
Generative Adversarial Networks



GANs: Saddle-Point Optimization

Saddle-Point Optimization: learn $G_{\theta_g}(\mathbf{z})$ and $D_{\theta_d}(\mathbf{x})$ jointly via the objective $\mathcal{J}(\theta_d, \theta_g)$:

$$\min_{\theta_g} \max_{\theta_d} \underbrace{\mathbb{E}_{p_{\text{data}}}\left[\log D_{\theta_d}(\mathbf{x})\right]}_{\text{likelihood of true data}} + \underbrace{\mathbb{E}_{p_{\mathbf{z}}(\mathbf{z})}\left[\log\left(1 - D_{\theta_d}(G_{\theta_g}(\mathbf{z}))\right)\right]}_{\text{likelihood of generated data}}$$



GANs: Optimal Discriminators

Claim: Given G_{θ_g} defining an implicit distribution $p_g = p(\mathbf{x} \mid \theta_g)$, the optimal descriminator is

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\text{g}}(\mathbf{x})}$$

Proof Sketch:

$$\begin{aligned}\mathcal{J}(\theta_d, \theta_g) &= \int_{\mathcal{D}} p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) d\mathbf{x} + \int_{\tilde{\mathcal{D}}} p(\mathbf{z}) \log\left(1 - D\left(G_{\theta_g}(\mathbf{z})\right)\right) d\mathbf{z} \\ &= \int_{\mathcal{D} \cup \tilde{\mathcal{D}}} p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x})) d\mathbf{x}\end{aligned}$$

Maximizing the integrand for all x is sufficient and gives the result

Optimal Discriminators Cont.

The integrand

$$h(D(\mathbf{x})) = p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x}))$$

is concave for $D(\mathbf{x}) \in (0, 1)$. We take the derivative and compute a stationary point in the domain:

$$\frac{\partial h(D(\mathbf{x}))}{\partial D(\mathbf{x})} = \frac{p_{\text{data}}(\mathbf{x})}{D(\mathbf{x})} - \frac{p_g(\mathbf{x})}{1 - D(\mathbf{x})} = 0$$

$$\Rightarrow D(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\text{g}}(\mathbf{x})}$$

This minimizes the integrand over the domain of the discriminator, completing the proof.

GANs: Jensen-Shannon Divergence and Optimal Generators

Given an optimal discriminator $D^*(\mathbf{x})$, the generator objective is

$$
\begin{aligned}
C\left(\theta_g\right) &= \mathbb{E}_{p_{\text{data}}}\left[\log D^*_{\theta_d}(\mathbf{x})\right] + \mathbb{E}_{p_g(\mathbf{x})}\left[\log\left(1 - D^*_{\theta_d}(\mathbf{x})\right)\right] \\
&= \mathbb{E}_{p_{\text{data}}}\left[\log \frac{p_{\text{data}}\left(\mathbf{x}\right)}{p_{\text{data}}\left(\mathbf{x}\right) + p_{\text{g}}(\mathbf{x})}\right] + \mathbb{E}_{p_g(\mathbf{x})}\left[\log \frac{p_g(\mathbf{x})}{p_{\text{data}}\left(\mathbf{x}\right) + p_{\text{g}}(\mathbf{x})}\right] \\
&\propto \underbrace{\frac{1}{2} KL\left(p_{\text{data}} \,\|\, \frac{\left(p_{\text{data}} + p_g\right)}{2}\right) + \frac{1}{2} KL\left(p_g \,\|\, \frac{\left(p_{\text{data}} + p_g\right)}{2}\right)}_{\text{Jensen-Shannon Divergence}}
\end{aligned}
$$

$C\left(\theta_g\right)$ achieves its global minimum at $p_g = p_{\text{data}}$ given an optimal discriminator!

Training Procedure

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

Discriminator updates

for k steps **do**

- Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

end for

Generator updates

- Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Problems with GANs

- Vanishing gradients: the discriminator becomes "too good" and the generator gradient vanishes.
- Non-Convergence: the generator and discriminator oscillate without reaching an equilibrium.
- Mode Collapse: the generator distribution collapses to a small set of examples.
- Mode Dropping: the generator distribution doesn't fully cover the data distribution.

A Better Cost Function

- We introduced the minimax cost function for the generator:

$$\mathcal{J}_G = \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$$

- One problem with this is saturation.
- Here, if the generated sample is really bad, the discriminator's prediction is close to 0 , and the generator's cost is flat.

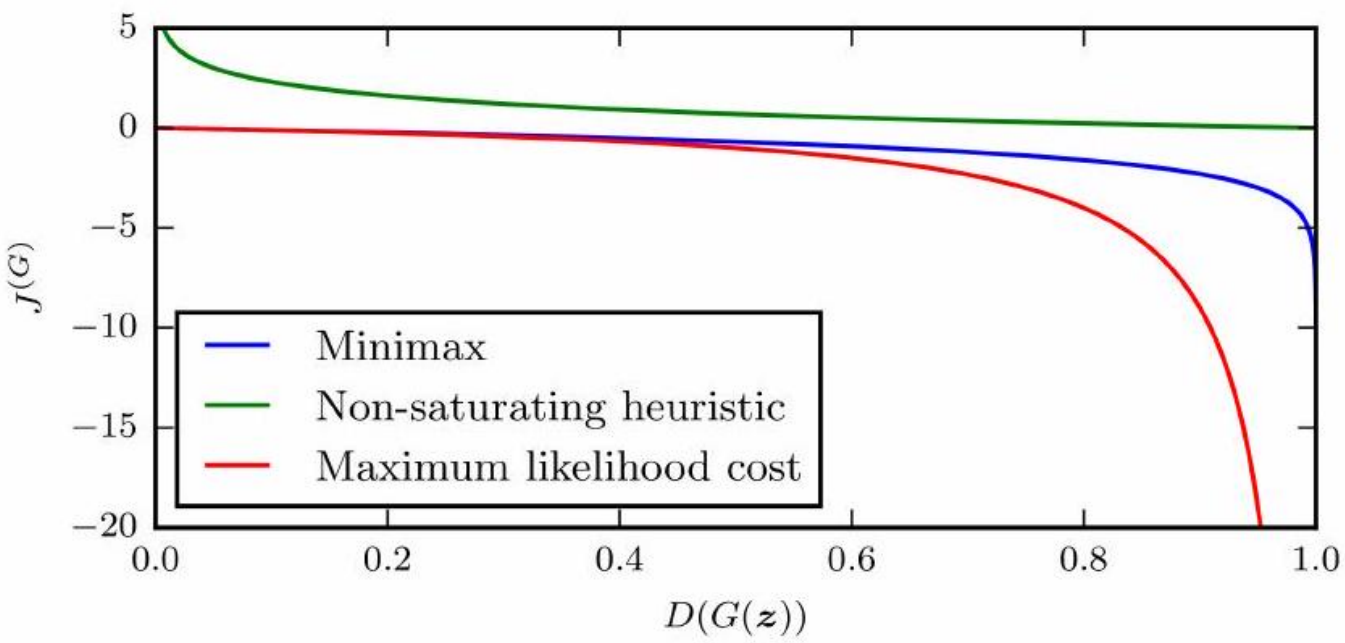
A Better Cost Function

- Original minimax cost:

$$\mathcal{J}_G = \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$$

- Modified non-saturating heuristic generator cost:

$$\mathcal{J}_G = \mathbb{E}_{\mathbf{z}}[-\log D(G(\mathbf{z}))]$$



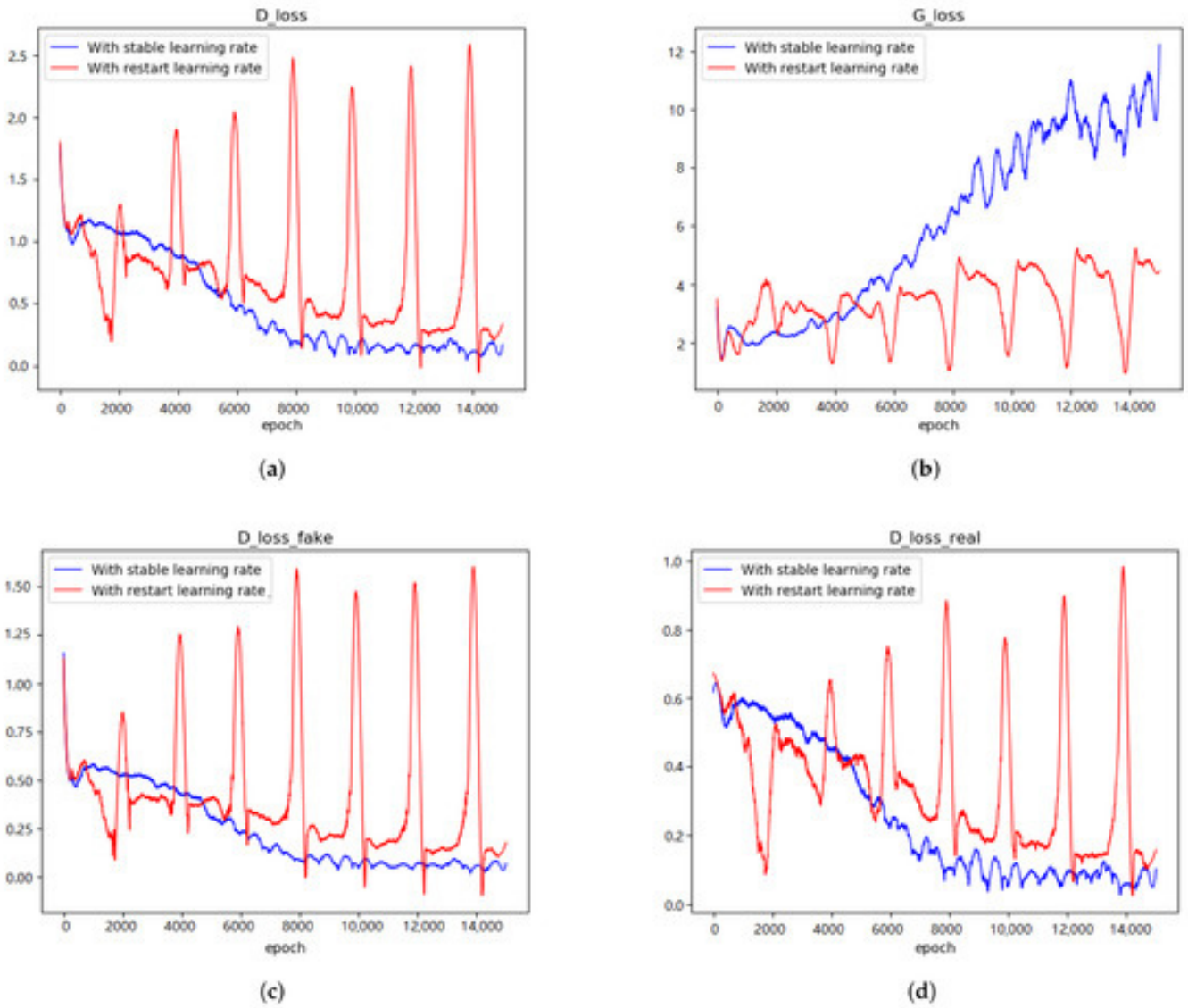
Problems: Addressing Vanishing Gradients

Solutions:

- Change Objectives: use the non-saturating heuristic objective, maximum-likelihood cost, etc.
- Limit Discriminator: restrict the capacity of the discriminator.
- Schedule Learning: try to balance training D_{θ_d} and G_{θ_g}.

Problems: Non-Convergence

Simultaneous gradient descent is not guaranteed to converge for minimax objectives.



Problems: Non-Convergence

- Goodfellow et al. only showed convergence when updates are made in the function space [2].
- The parameterization of D_{θ_d} and G_{θ_g} results in highly non-convex objective.
- In practice, training tends to oscillate - updates "undo" each other.

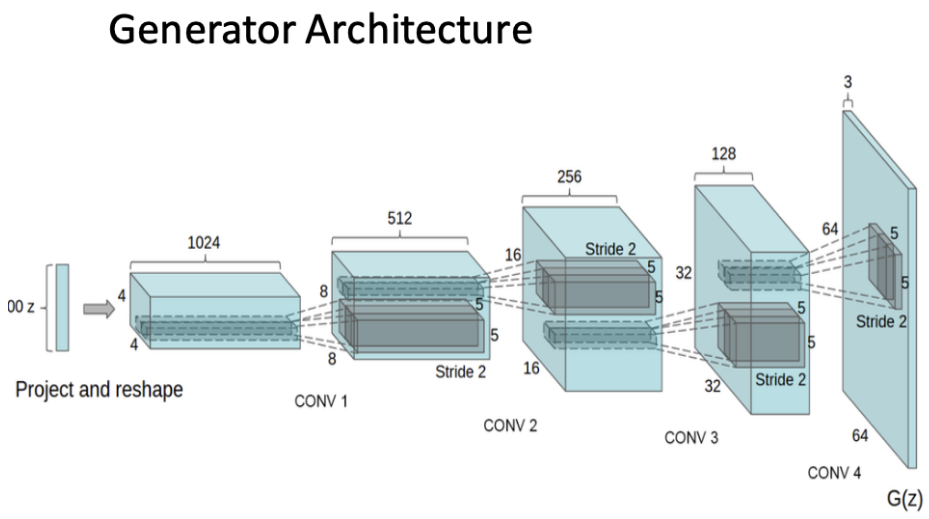
Addressing tricks:

- Use DCGAN
- Use stability tricks from RL
- Use the ADAM Optimizer
- Use SGD for discriminator and ADAM for generator

DC-GAN

- Most modern architectures are based on the Deep Convolutional GAN (DC-GAN).
- GAN Zoo: https://github.com/hindupuravinash/the-gan-zoo

Deep Convolutional GANs (DCGANs)



Key ideas:

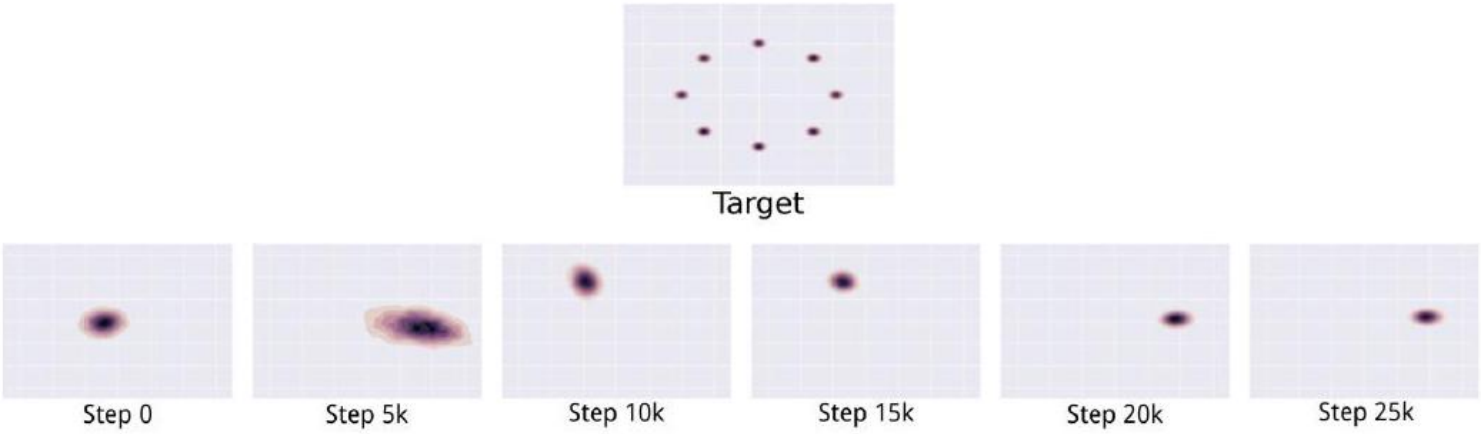
- Replace FC hidden layers with Convolutions
  - **Generator:** Fractional-Strided convolutions
- Use Batch Normalization after each layer
- **Inside Generator**
  - Use ReLU for hidden layers
  - Use Tanh for the output layer

Problems: Mode Collapse and Mode Dropping

One Explanation: SGD may optimize the max-min objective

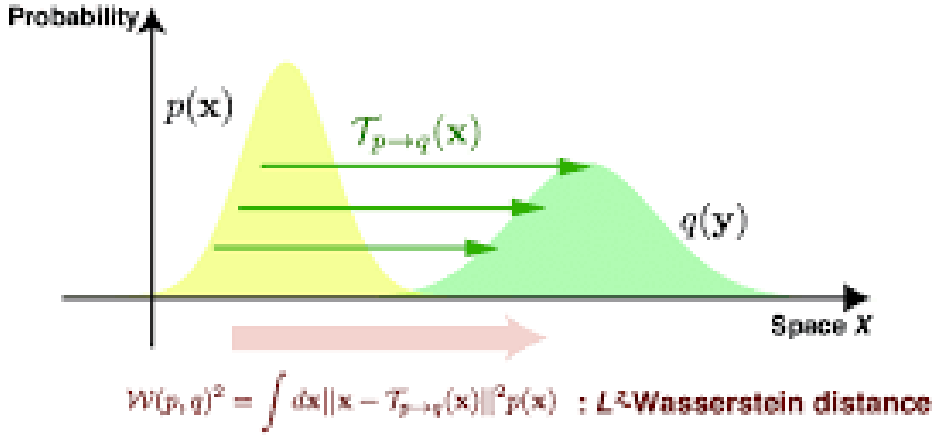
$$\max_{\theta_d} \min_{\theta_g} \mathbb{E}_{p_{\text{data}}} \left[\log D_{\theta_d}(\mathbf{x})\right] + \mathbb{E}_{p_{\mathbf{z}}(\mathbf{z})} \left[\log \left(1 - D_{\theta_d}\left(G_{\theta_g}(\mathbf{z})\right)\right)\right]$$

Intuition: the generator maps all $\mathbf{z}$ values to the $\hat{\mathbf{x}}$ that is mostly likely to fool the discriminator.



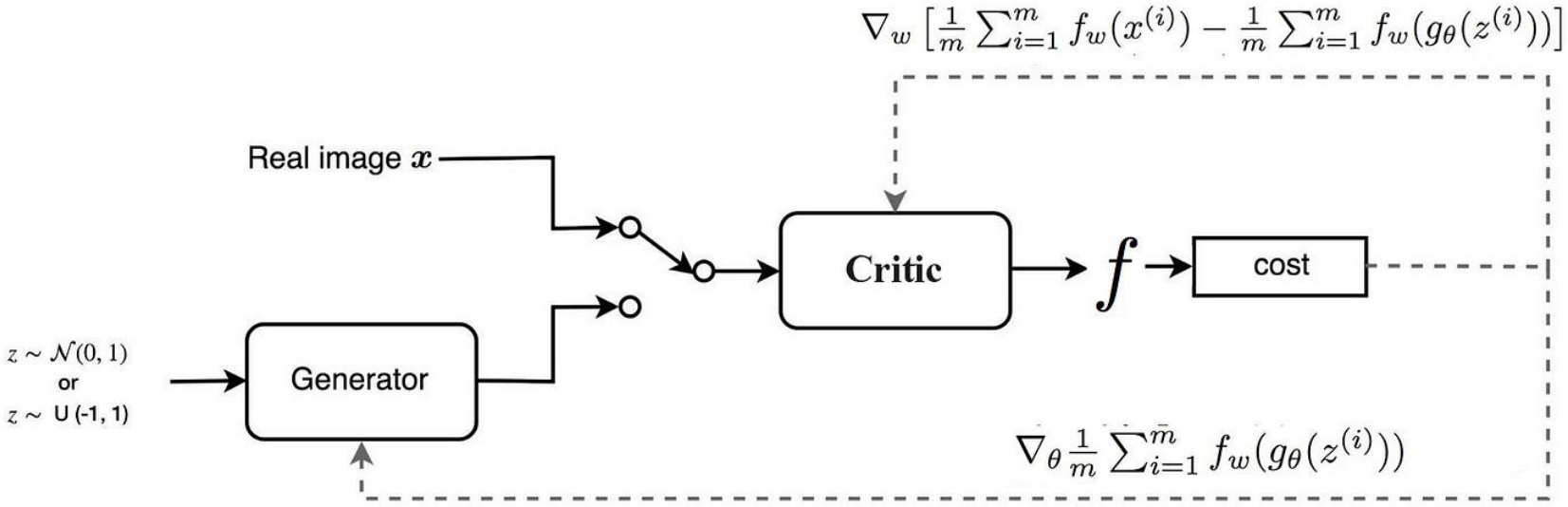
Wasserstein GANs

The Wasserstein distance (or Earth Mover's distance) intuitively represents the minimum "cost" required to transform one distribution into another.



WGAN

WGANs


$\nabla_w \left[\frac{1}{m}\sum_{i=1}^{m} f_w(x^{(i)}) - \frac{1}{m}\sum_{i=1}^{m} f_w(g_\theta(z^{(i)}))\right]$
Real image x
Critic
f
cost
$z \sim \mathcal{N}(0,1)$
or
$z \sim$ U (-1, 1)
Generator
$\nabla_\theta \frac{1}{m}\sum_{i=1}^{m} f_w(g_\theta(z^{(i)}))$

Reinforcement Learning Recap

Introduction to Generative AI
- Autoencoders
- Generative Adversarial Networks (GAN)

Applications in Finance

Challenges of Generative AI in Finance

Applications of Generative AI in Finance

- Synthetic Data Generation:
  - Create realistic financial time series, order book data, etc.
- Scenario Generation for Stress Testing:
  - Generate diverse, plausible market scenarios to test risk management strategies.
- Data Augmentation:
  - Enhance limited or imbalanced datasets for credit scoring, fraud detection, etc.
- Market Microstructure Simulation:
  - Simulate realistic order book dynamics for training algorithmic trading strategies. $\Rightarrow$ MARS (MSFT Asia)
- Option Pricing and Volatility Modeling:
  - Model complex pricing surfaces and volatility smiles using non-linear dependencies.
- Portfolio Optimization & Risk Management
  - Allows portfolio managers to understand how assets might co-move in extreme scenarios.

Why Use Generative AI in Finance?

- Enhance Risk Management:
  - Identify anomalies and extreme events, aiding in risk monitoring and decision making.
- Overcome Data Scarcity:
  - Generate additional high-quality data to support training and stress testing.
- Improve Model Robustness:
  - Augmented data enhances generalization and robustness of predictive models.
- Enable Advanced Simulation:
  - Create controlled, high-fidelity environments to simulate market dynamics.
- Capture Non-Linear Dynamics:
  - Learn complex relationships in financial data that traditional models may miss.

Reinforcement Learning Recap

Introduction to Generative AI
- Autoencoders
- Generative Adversarial Networks (GAN)

Applications in Finance

Challenges of Generative AI in Finance

Overview

Group discussion

Overview

Group discussion

- Evaluation in generative AI for finance is multifaceted.

1. Multidimensional Evaluation Criteria and Lack of Standardized Metrics

- Quality of Synthetic Data:
  - Reflects complex time-series patterns and dependencies.
  - Requires metrics that capture statistical and domain-specific nuances.
- Diversity vs. Fidelity Trade-off:
  - Balancing realistic samples with diverse outputs is challenging.
- Absence of Benchmarks:
  - No widely accepted metrics similar to FID or Inception Score for financial data.
- Complex Financial Objectives:
  - Evaluations need to consider predictive performance, risk sensitivity, and economic interpretability.

2. Integration with Downstream Tasks

- Scenario Testing:
  - Assess impact on portfolio optimization and risk management.
  - Requires extensive backtesting and simulation.
- Risk Assessment:
  - Must capture tail risks and extreme market events.
- Regulatory & Compliance:
  - Evaluations must ensure adherence to legal and ethical standards.

3. Dynamic and Non-Stationary Environments

- Market Evolution:
  - Models must adapt to rapid changes and different market regimes.
- Robustness to Shocks:
  - Validating performance during rare market crises is non-trivial.

4. Computational Challenges

- High-Dimensional Data:
  - Increased computational burden in training and evaluation.
- Simulation-Based Testing:
  - Requires significant resources to simulate diverse market conditions.